PARKORA

Full Technical Stack Document

Event & Parking Reservation System

Technology versions, frameworks, database, languages, APIs, libraries, and development tools used by the project.

Verified project baseline: ASP.NET Core Web API targeting .NET 8, vanilla HTML/CSS/JavaScript frontend, Entity Framework Core 8, and Microsoft SQL Server LocalDB.

1. Technical Stack at a Glance

Area	Technology / Version
.NET Version	.NET 8 (Target Framework: net8.0)
Frontend	HTML5, CSS3, Vanilla JavaScript (no React/Angular/Vue)
Backend	ASP.NET Core Web API using Microsoft.NET.Sdk.Web
Database	Microsoft SQL Server / SQL Server LocalDB - database: Eventparkingdb
Data Access	Entity Framework Core 8.0.8 with SQL Server provider
Authentication	JWT Bearer authentication + BCrypt password hashing
API Documentation	Swagger / OpenAPI via Swashbuckle.AspNetCore 6.6.2
Frontend-Backend Integration	Fetch API + JSON over REST endpoints
Primary IDE / DB Tool	Visual Studio 2022 + SQL Server Management Studio (SSMS)
Version Control	Git and GitHub

2. .NET and Backend Technology

Target framework: .NET 8 (net8.0). The project is an ASP.NET Core Web API application using the Microsoft.NET.Sdk.Web project SDK.

```
<TargetFramework>net8.0</TargetFramework>
```

Backend architecture

- ASP.NET Core Controllers receive HTTP requests and return JSON responses.
- Service interfaces and service classes contain business logic for authentication, events, bookings, payments, feedback, seats, parking, notifications, dashboards, and holds.
- Repository interfaces and repository classes isolate data-access operations from business logic.
- Dependency Injection is provided by the built-in ASP.NET Core service container using scoped registrations.
- Custom exception middleware provides centralized API error handling.
- ASP.NET Core serves the frontend directly from wwwroot using UseDefaultFiles and UseStaticFiles.

Backend framework features in use

Feature	Implementation
Controllers / Routing	ASP.NET Core MVC controller routing via MapControllers().
Dependency Injection	Built-in ASP.NET Core DI with AddScoped registrations.
Configuration	appsettings.json for database connection and JWT settings.
Middleware	HTTPS redirection, static files, CORS, authentication, authorization, and custom exception middleware.
Serialization	System.Text.Json; reference cycles ignored for EF navigation objects.
Security	JWT validation for issuer, audience, lifetime, and signing key.

3. Frontend Technology

Frontend approach: Responsive vanilla web frontend stored under wwwroot. The project does not require a JavaScript framework such as React, Angular, or Vue.

Technology	Use in Parkora
HTML5	index.html defines the application views, forms, navigation, modals, booking UI, feedback UI, and admin UI.
CSS3	site.css provides the Parkora visual design, responsive layouts, modal styling, seat/parking states, cards, tables, and media queries.
JavaScript	app.js manages UI state, login/session state, API calls, event rendering, booking selection, payment, feedback, notifications, and admin behavior.
Fetch API	Native browser fetch() is used to communicate with the ASP.NET Core REST API.
JSON	Request and response bodies are exchanged in JSON format.
Web Storage	localStorage stores the customer/admin JWT token, user details, and theme preference in the browser.

4. Database and Data Access

Database: Microsoft SQL Server. In the current development configuration the project connects to SQL Server LocalDB and uses the database name Eventparkingdb.

```
Server=(localdb)\MSSQLLocalDB;Database=Eventparkingdb;Trusted_Connection=True;TrustServerCertificate=True;
```

Entity Framework Core

- Entity Framework Core 8.0.8 is used as the Object-Relational Mapper (ORM).
- Microsoft.EntityFrameworkCore.SqlServer 8.0.8 provides the SQL Server database provider.
- Microsoft.EntityFrameworkCore.Tools 8.0.8 supports migrations and Package Manager Console commands such as Update-Database.
- AppDbContext represents the database session and defines the entity model, relationships, constraints, and seed data.

Core database entities

Customers, Venues, EventCategories, Events, Seats, ParkingSlots, Bookings, BookingSeats, ParkingReservations, Payments, Feedbacks, Notifications, Seat/Hold records.

5. Programming Languages

Language	Purpose
C#	Backend controllers, services, repositories, models, DTOs, middleware, EF Core configuration, authentication, and business logic.
JavaScript	Frontend application logic, Fetch API calls, DOM updates, booking state, form handling, and client-side interaction.
HTML	Page structure, forms, navigation, modal content, and semantic UI markup.
CSS	Visual design, responsive layouts, animations, states, seat/parking UI, and theme styling.
SQL	Database inspection, manual verification/maintenance, and EF Core-generated SQL through migrations.

6. APIs and Services Used

API / Service	Role in the System
REST API	ASP.NET Core exposes resource-based HTTP endpoints for Auth, Customers, Events, Categories, Venues, Seats, Parking Slots, Bookings, Payments, Feedback, Notifications, and Dashboards.
Fetch API	The browser uses native fetch() for all frontend-to-backend communication.
JWT Authentication Service	Login returns a JWT token that is sent in the Authorization: Bearer header for protected API requests.
Swagger / OpenAPI	Interactive API documentation and endpoint testing are available during development.
CORS Service	A configured FrontendPolicy controls allowed browser origins, headers, and methods.
Static File Service	ASP.NET Core serves index.html, site.css, and app.js directly from wwwroot.
Payment Service	The project implements a simulated internal payment workflow; no real external payment gateway is required.
Feedback Service	Authenticated users submit booking-related ratings/comments and feedback is exposed through project API endpoints.
Notification Service	Booking/payment/cancellation notifications are stored and retrieved through the project backend.
Hold / Reservation Logic	Backend services manage temporary seat hold/reservation and availability logic as part of booking protection.

External service note: The inspected project configuration does not require a third-party payment gateway or external cloud API. Payment and notification behavior is implemented inside the application for the student project.

7. NuGet Packages and Libraries

Package	Version	Purpose
BCrypt.Net-Next	4.0.3	Secure password hashing and password verification.

Package	Version	Purpose
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.8	JWT Bearer authentication middleware.
Microsoft.EntityFrameworkCore	8.0.8	Core ORM and DbContext functionality.
Microsoft.EntityFrameworkCore.SqlServer	8.0.8	SQL Server provider for Entity Framework Core.
Microsoft.EntityFrameworkCore.Tools	8.0.8	EF Core migration/design-time tooling.
Swashbuckle.AspNetCore	6.6.2	Swagger/OpenAPI generation and Swagger UI.

8. Other Tools and Technologies

Tool / Technology	Use
Visual Studio 2022	Primary IDE for ASP.NET Core development, debugging, NuGet packages, EF Core migrations, and project execution.
SQL Server Management Studio (SSMS)	Database inspection, SQL queries, validation of Customers/Bookings/Payments/Feedbacks and other tables.
Swagger UI	Interactive testing and documentation of REST endpoints.
Git	Local source-control workflow: status, add, commit, branch, merge, and push.
GitHub	Remote repository, branch collaboration, and team source-code sharing.
Browser Developer Tools	Network/Console inspection for HTTP status codes such as 401, 403, 409, and 500 and for frontend JavaScript debugging.
NuGet	Package management for EF Core, JWT, BCrypt, and Swagger dependencies.
HTTPS / Development Certificate	ASP.NET Core development HTTPS profile for secure local API/browser communication.

9. Application Architecture

High-level request flow

```
Browser (HTML/CSS/JS) -> Fetch API / JSON -> ASP.NET Core Controller -> Service -> Repository -> EF Core -> SQL Server
```

Layer	Technology / Responsibility
Presentation Layer	HTML, CSS, JavaScript in wwwroot.
API Layer	ASP.NET Core Controllers and REST endpoints.
Business Logic Layer	Service interfaces and service implementations.
Data Access Layer	Repository interfaces/implementations plus Entity Framework Core.

Persistence Layer	Microsoft SQL Server / LocalDB.
Security Layer	BCrypt password hashing, JWT authentication, role-based authorization, HTTPS, and CORS.

10. Final Technical Stack Summary

Parkora is a full-stack .NET 8 web application. Its frontend uses HTML5, CSS3, and vanilla JavaScript; its backend uses ASP.NET Core Web API with a layered Controller-Service-Repository architecture; Entity Framework Core 8.0.8 connects to Microsoft SQL Server; JWT and BCrypt provide authentication/password security; and Swagger/OpenAPI, Git/GitHub, Visual Studio 2022, and SSMS support development, testing, and collaboration.

Document prepared from the project configuration and source structure.